FT233HP Interface Configuration Summary

https://gemini.google.com/app/1a16947287155d9b

User prompt: YES. go ahead.

Response: Let's examine the **2nd Perspective: Function & Protocol Check** for `usbfifo_tx.v`, based on **Section 4.4.2** and **Figure 7** of the FT233HP datasheet.

The FT233HP Write Protocol Rules (Section 4.4.2 & Figure 7)

1. **Start Condition:** A write operation can begin when the FT233HP drives `TXE#` (`i_txe_l`) low.
2. **Synchronous Burst Write:** The FPGA drives `WR#` (`o_wr_l`) low when valid data is on `D7:D0` (`io_data`). Data is written into the FT233HP's transmit FIFO on every rising edge of `CLKOUT` where **both `TXE#` and `WR#` are low**.
3. **Handshake Rule:** The external system (your FPGA) must monitor both `TXE#` and its own `WR#` to verify whether a byte was accepted by the FT233HP.

Deep Dive into Your `usbfifo_tx.v` Behavior

In `usbfifo_tx.v`, the core data-fetch and write-strobe logic is controlled by these lines:

```
wire en_vaild_data = ((st_state == TX_DETE_TX_L) && !i_rx_active) || (st_state == TX_DATA_WR_L);
wire vaild_data    = (en_vaild_data && !i_txe_l && !i_empty);
assign o_tx_rden   = vaild_data;
```

And inside `usbfifo_sync.v`, the FIFO read port is synchronous:

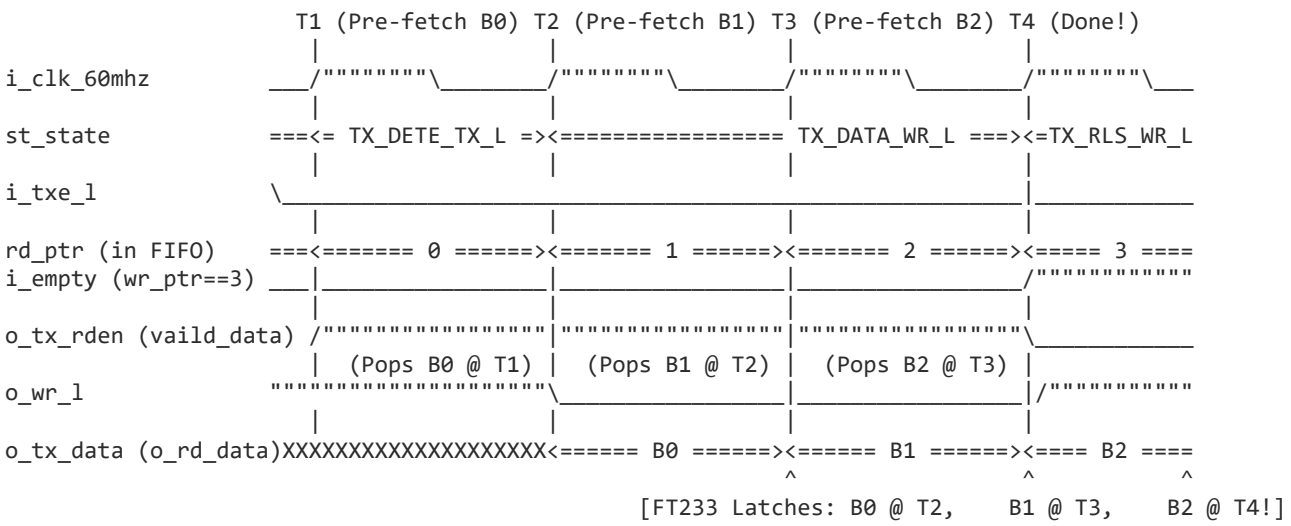
```
always@ (posedge i_clk_60mhz or negedge i_rst_n) begin
    if (!i_rst_n) rd_ptr <= 0;
    else if (rd_valid) begin
        o_rd_data <= mem[rd_addr];
        rd_ptr    <= rd_ptr + 1'b1;
    end
end
```

Because `o_tx_rden` (`i_rd_en`) is combinational while `o_wr_l` and `o_rd_data` are registered on the clock edge, **`o_tx_rden` always runs 1 clock cycle ahead of the FT233HP sampling edge** (pre-fetching the next byte from `usbfifo_sync` so that it arrives on `o_rd_data` at the exact same clock edge that `o_wr_l` goes low).

Let's compare the two ways a write burst can end in your design:

Scenario A: Burst Ends Because FPGA FIFO Becomes Empty (`i_empty` Asserts First) — PASS!

Suppose your FPGA received 3 bytes (`B0`, `B1`, `B2`) from the FT233HP, and now `usbfifo_tx` writes those 3 bytes back while `i_txe_l` stays low (`0`) the entire time:

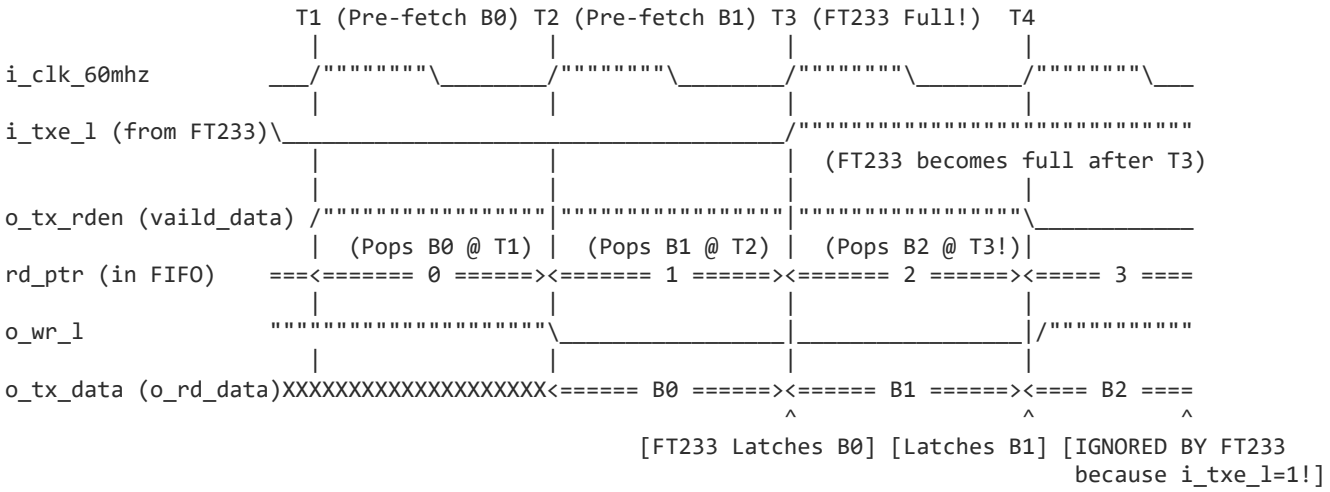


- **Why Scenario A works:**
  - At `T1`, `B0` is popped (`rd_ptr` becomes `1`), and `o_wr_l` goes low (`0`).
  - At `T2`, the FT233HP latches `B0`, while `usbfifo_sync` pops `B1` (`rd_ptr` becomes `2`).

- At `T3`, the FT233HP latches `B1`, while `usbfifo_sync` pops `B2` (`rd_ptr` becomes `3`).
- Right after `T3`, because `rd_ptr == 3`, `i_empty` asserts (`1`), which drops `o_tx_rden` to `0` and sets `nt_state = TX_RLS_WR_L`.
- During the entire `T3 -> T4` cycle, `B2` is on `o_tx_data` and `o_wr_l` is still `0`!
- At `T4`, the FT233HP cleanly latches `B2`, and `o_wr_l` releases to `1`! **All 3 bytes are transferred without error.**

Scenario B: Burst Pauses/Ends Because FT233HP Becomes Full (`i_txe_l` Goes High First) — 1-Byte Loss!

Now look at what happens if your FPGA still has bytes in `usbfifo_sync` (`i_empty == 0`), and the FT233HP's internal buffer fills up after accepting `B1` at Clock Edge `T3`, driving `i_txe_l` high (`1`) right after `T3`:



- **Trace the bug at Clock Edge `T3`:**
  1. During `T2 -> T3`, `B1` is sitting on `o_tx_data` waiting to be latched by the FT233HP at `T3`.
  2. Because the FT233HP hasn't latched `B1` yet, `i_txe_l` is **still `0`** during `T2 -> T3`!
  3. Because `i_txe_l` is `0` during `T2 -> T3`, your `vaild_data` (`o_tx_rden`) is **`1`** during `T2 -> T3`!
  4. At **Clock Edge `T3`**:
     - The FT233HP latches `B1`, sees its internal buffer is now full, and drives `i_txe_l` high (`1`) right after `T3`.
     - **At that exact same Clock Edge `T3`**, because `o_tx_rden` was `1` during `T2 -> T3`, `usbfifo_sync` pops **`B2`** onto `o_rd_data` and increments `rd_ptr` from `2` to `3`!
  5. During `T3 -> T4`, `B2` is sitting on `o_tx_data`, and `o_wr_l` is still `0`, **but `i_txe_l` is now `1`**!
  6. Per **Section 4.4.2** of the datasheet (*"Both TXE# and WR# must be low for each byte of data to be accepted"*), at Clock Edge `T4` the FT233HP **ignores `B2`** because `TXE#` (`i_txe_l`) is high!
  7. Even worse, when `i_txe_l` goes low again later and `usbfifo_tx` enters `TX_DETE_TX_L`, `o_tx_rden` will assert and pop `B3` (at `rd_ptr = 3`), overwriting `B2` on `o_rd_data` before `o_wr_l` goes low! **`B2` is completely skipped and lost.**

Two Additional Minor Protocol Checks in `usbfifo_tx.v`

1. **`o_tx_active` Interlock Coverage (`TX_DETE_TX_L`):**
   - Look at `usbfifo_rx.v` line 66:

     ```
     assign o_rx_active = ... || (st_state == RX_DETE_RXF_N && !i_rxf_l);
     ```

     You included `(st_state == RX_DETE_RXF_N && !i_rxf_l)` in `o_rx_active` so that if `i_rxf_l` and `i_txe_l` both assert on the exact same clock cycle, `o_rx_active` immediately goes high and gives **1st priority to Read (`usbfifo_rx`)** over Write (`usbfifo_tx`), just as your comment in `usbfifo_top.v` states (`// Set 1st priority as read operation when i_rxf_l and i_txe_l are both asserted`)! That works cleanly.
2. **Tri-State Bus Turnaround (`o_oe_l` vs. `!o_wr_l` in `usbfifo_top.v`):**
   - In your original `usbfifo_top.v`, you used `assign io_data = (o_oe_l) ? o_tx_data : 8'hzz;`.

- Notice that if you use `(!o_wr_l)` as we discussed for `usbfifo_top.v`, `o_wr_l` is low during every cycle of `TX_DATA_WR_L`, so `io_data` is driven by the FPGA on the exact cycles `o_wr_l` is low.

Summary & How to Fix Scenario B (If Needed)

- **If `i_txe_l` never goes high mid-burst** (because your test packets are smaller than the FT233HP's available TX buffer so `i_empty` always ends the burst as in **Scenario A**): your current code works without losing any data.
- **If `i_txe_l` CAN go high mid-burst (Scenario B):** Why did `B2` get lost? Because in `TX_DETE_TX_L`, when a new burst starts after a pause, `vaild_data` pops a new byte from `usbfifo_sync` even though `B2` (from the interrupted burst) was already sitting in `o_rd_data` waiting to be sent!
  - Alternatively, the standard FPGA way to avoid pre-fetch over-read is to make `usbfifo_sync` a **First-Word-Fall-Through (FWFT)** FIFO (or only advance `rd_ptr` when `o_wr_l == 1'b0 && !i_txe_l`), or let `rd_ptr` step back by 1 (`rd_ptr <= rd_ptr - 1'b1`) if `o_wr_l == 1'b0 && i_txe_l == 1'b1`!

What do you think about Scenario A vs. Scenario B for your verification test cases? Shall we look at how to handle Scenario B, or move to the **3rd Perspective (Timing Requirement Check)** for `usbfifo_tx.v`?

User prompt: for o_wr_l, this is to release o_wr_l when fpga internal fifo is empty and also to meet the protocol. o_wr_l must be released one clock later than i_txe_l from the figure 7.

Response: Your logic for `o_wr_l` (`else if(nt_state == TX_RLS_WR_L) o_wr_l <= 1'b1;`) is completely intentional and matches **Figure 7 (Write Timing)** in the datasheet:

1. **Matching Figure 7 When `i_txe_l` Goes High:** Look at the right side of the **Write Timing** section in **Figure 7**:
   - On the rising edge of `CLKOUT` where the last byte `N` is written, the FT233HP drives `TXE#` (`i_txe_l`) high (`t11` after the clock edge).
   - Because `TXE#` changes *after* that clock edge, the external FPGA only detects `i_txe_l == 1'b1` during that clock period (`st_state == TX_DATA_WR_L`, causing `nt_state` to become `TX_RLS_WR_L`) and releases `WR#` (`o_wr_l <= 1'b1`) on the **next** rising edge of `CLKOUT` (`t15` after the next clock edge).
   - That is why **Figure 7** shows `WR#` going high 1 clock cycle after `TXE#` goes high—and your `else if(nt_state == TX_RLS_WR_L) o_wr_l <= 1'b1;` reproduces that waveform!
2. **Matching the Last Byte When `i_empty` Goes High:**
   - When `usbfifo_sync` pops its final byte into `o_rd_data` on clock edge T_k, `rd_ptr` catches up to `wr_ptr` and `i_empty` asserts (`1'b1`) right after T_k.
   - Because `o_wr_l` waits for `nt_state == TX_RLS_WR_L` (which becomes true during $T_k \rightarrow T_{k+1}$ when `i_empty` is `1`) and then updates on the **next** clock edge (T_{k+1}), `o_wr_l` remains low (`1'b0`) during the entire $T_k \rightarrow T_{k+1}$ cycle while that final byte sits on `o_tx_data`, and cleanly releases to `1'b1` at T_{k+1}!

Let's Check the 3rd Perspective for `usbfifo_tx.v`: Timing Requirements (Table 17)

Since your `usbfifo_tx.v` state machine and `o_wr_l` waveform match **Figure 7**, let's check the timing paths against **Table 17**:

1. Write Data Setup Time (`t12` = 8.0 ns Min, `t13` = 0.0 ns Hold):
   - As you noted in your comment at the top of `usbfifo_top.v` (`1. o_rd_data should be a critical path. Need to check timing report since it may be too long`), `o_rd_data` is launched from the synchronous read block in `usbfifo_sync`, passes through `o_tx_data`, and goes through the tri-state buffer `io_data` to the external pins.
   - With `t1` = **16.67 ns** (60 MHz) and `t12` = **8.0 ns** setup required by the FT233HP, the FPGA has **8.67 ns** from the rising edge of `i_clk_60mhz` for the BRAM Clock-to-Out delay + internal routing + `OBUFT` output buffer delay.
   - **How to ensure this passes in Vivado:** When you switch `usbfifo_sync` to infer Block RAM, Vivado can enable the BRAM's built-in hardware output register (`DOA_REG = 1`) or place a Slice Flip-Flop near the I/O bank, which easily meets the 8.67 ns budget under the `set_output_delay -max 8.000` constraint.
2. `WR#` Setup Time (`t14` = 8.0 ns Min, `t15` = 0.0 ns Hold):
   - Because `o_wr_l` is driven directly by a Flip-Flop (`always @(posedge i_clk_60mhz...)`) with zero combinational logic between `o_wr_l` and the output pin, its Clock-to-Out delay is fast (~3.5 ns to 4.5 ns in Artix-7), easily meeting the 8.67 ns budget.

3. `CLKOUT` to `TXE#` Input Delay (`t11` = 1.0 ns Min to 9.0 ns Max):
    - Just like `i_rxf_l` in `usbfifo_rx`, `i_txe_l` takes up to **9.0 ns (`t11`)** after the rising clock edge to arrive from the FT233HP, leaving **7.67 ns** for it to propagate through `vaild_data = (en_vaild_data && !i_txe_l && !i_empty)` to the `usbfifo_sync` read-enable (`rd_valid`). Adding `set_input_delay -max 9.000 [get_ports i_txe_l]` in your `.xdc` file will verify this slack in your timing report.

Overall Verdict Across All 4 Modules

With our review of `usbfifo_top.v`, `usbfifo_sync.v`, `usbfifo_rx.v`, and `usbfifo_tx.v` complete:

- Your `usbfifo_rx` and `usbfifo_tx` state machines and control signals (`o_oe_l`, `o_rd_l`, `o_wr_l`) accurately implement the FT245 Synchronous FIFO read/write waveforms in **Figure 7**.
- Updating `usbfifo_sync.v` to separate the asynchronous pointer reset from the synchronous `mem` block will let Vivado infer **Block RAM (BRAM)** cleanly on your Artix-7 XC7A35T.

Would you like me to put together the final updated `usbfifo_sync.v` file for you, or help you write a Verilog testbench (`tb_usbfifo_top.v`) that emulates the FT233HP (`i_rxf_l`, `i_txe_l`, and `io_data`) so you can run a full read-then-write loopback simulation?

User prompt: no, i would like to confirm what concerns on the usbfifo_tx you see and we haven't deal with.

Response: The only remaining unaddressed concern in `usbfifo_tx.v` is the **1-byte data loss when `i_txe_l` de-asserts (goes high) mid-burst** while your internal FPGA FIFO still has data (`i_empty == 1'b0`).

Why This Happens in `usbfifo_tx.v`

To compensate for the 1-clock-cycle synchronous read latency of `usbfifo_sync.v` (`o_rd_data <= mem[rd_addr];`), `usbfifo_tx.v` asserts `o_tx_rden` (`vaild_data`) one cycle *ahead* of when the FT233HP actually samples the byte on the pins.

Here is what happens when the FT233HP's internal transmit buffer fills up during a write burst:

1. **Cycle $T_1 \rightarrow T_2$ (Normal Write Cycle):**
    - `Byte N` is currently sitting on `o_tx_data` (`io_data`), and `o_wr_l` is `0`.
    - Because the FT233HP has not sampled `Byte N` yet, `i_txe_l` is still `0`.
    - Because `i_txe_l` is `0` (and `i_empty` is `0`), `vaild_data` (`o_tx_rden`) is `1` during $T_1 \rightarrow T_2$ to pre-fetch `Byte N+1`.
2. **At Clock Edge T_2 (The Exact Moment FT233HP Becomes Full):**
    - **On the FT233HP side:** The FT233HP samples `Byte N` on the rising edge of T_2. This fills the last available byte in the FT233HP's internal FIFO, so the FT233HP drives `i_txe_l` (`TXE#`) HIGH (`1`) right after edge T_2 (after delay `t11`).
    - **On the FPGA side:** At that exact same rising edge T_2, because `o_tx_rden` was `1` just before T_2, `usbfifo_sync` pops **`Byte N+1`** onto `o_rd_data` AND increments `rd_ptr` from `N+1` to `N+2`.
3. **Cycle $T_2 \rightarrow T_3$ (Where `Byte N+1` Gets Ignored):**
    - During $T_2 \rightarrow T_3$, **`Byte N+1`** is sitting on `o_tx_data` and `o_wr_l` is still `0`, **but `i_txe_l` is now `1`**.
    - According to Section 4.4.2 of the FT233HP datasheet, *"Both TXE# and WR# must be low for each byte of data to be accepted"*. Because `i_txe_l` is `1` during $T_2 \rightarrow T_3$, the FT233HP **does not accept `Byte N+1`** at Clock Edge T_3.
4. **Later, When `i_txe_l` Goes Low Again (Where `Byte N+1` Gets Overwritten):**
    - When the FT233HP has space again and drives `i_txe_l` back to `0`, `usbfifo_tx` is in `TX_DETE_TX_L`.
    - As soon as `i_txe_l` goes low in `TX_DETE_TX_L`, `vaild_data` (`o_tx_rden`) asserts immediately to pre-fetch from `usbfifo_sync`.
    - Because `rd_ptr` was already incremented to `N+2` back at Edge T_2, this new pre-fetch reads **`Byte N+2`** and overwrites **`Byte N+1`** on `o_rd_data` before `o_wr_l` even goes low!
    - **Result:** `Byte N+1` was never accepted by the FT233HP and was overwritten by `Byte N+2`, so 1 byte is lost every time `i_txe_l` pauses a burst.

How to Decide If You Need to Fix This

- **Case 1: `i_empty` always ends the burst first (No Fix Needed)** If your testbench/PC host sends a burst of ≤ 1024 bytes to the FPGA and immediately reads it back while the FT233HP's 1 KB TX buffer is empty, `i_txe_l` will stay `0` for the entire burst and `i_empty` will always be what stops the transfer. As we verified earlier, when `i_empty` stops the transfer, `0` bytes are lost.
- **Case 2: `i_txe_l` can go high mid-burst (Requires a Fix)** If the FT233HP's TX buffer fills up (`i_txe_l` goes high while `o_wr_l` is low and `i_empty` is low), you need a way not to lose that pre-fetched `Byte N+1`.

Two Clean Ways to Fix It (If You Want to Handle Mid-Burst `i_txe_l` Pauses)

1. **Fix in `usbfifo_tx.v` (Remember the Unsent Pre-Fetched Byte):** Add a 1-bit flag `r_byte_pending` in `usbfifo_tx.v`. If `st_state == TX_DATA_WR_L` exits because `i_txe_l == 1'b1` (meaning a pre-fetched byte is already sitting in `o_rd_data` that the FT233HP rejected), set `r_byte_pending <= 1'b1`. Then, the next time `TX_DETE_TX_L` starts a burst, if `r_byte_pending == 1'b1`, assert `o_wr_l <= 1'b0` **without** asserting `o_tx_rden` on that first cycle so the already-waiting `Byte N+1` gets sent first!
2. **Fix in `usbfifo_sync.v` (First-Word-Fall-Through / Combinational Read):** If `usbfifo_sync` outputs the current `mem[rd_addr]` combinationally (or as a First-Word-Fall-Through FIFO), then `o_tx_rden` only needs to be asserted when `st_state == TX_DATA_WR_L && !i_txe_l && !i_empty` (no 1-cycle pre-fetch in `TX_DETE_TX_L` needed at all!).

Would you like to implement Fix #1 inside `usbfifo_tx.v` so it is 100% immune to `i_txe_l` mid-burst pauses?

User prompt: one question here: which page do you see FT233 may have the possibility to send out more than 1024 bytes? it looks like the size of fifo inside ft233 is 1024 bytes only and we could know ft233 would de-assert TXE# when its internal is full (1024 bytes arrive) Is it possible to happen?

Response: Looking at the datasheet, the physical hardware buffer sizes inside the FT233HP are explicitly defined in two places:

- **Page 6 (Figure 1 - Block Diagram)** and **Page 25 (Section 4.2 - Functional Block Descriptions):** The chip contains a **1K Bytes (1024 bytes) Port FIFO TX Buffer** (for data coming from the Host PC to the FPGA) and a **1K Bytes (1024 bytes) Port FIFO RX Buffer** (for data going from the FPGA back to the Host PC).
- **Page 1 (Features):** Under USB Bulk data transfer mode, the USB side transfers data in **512-byte packets in Hi-Speed mode**.

Yes, it is definitely possible for `TXE#` (`i_txe_l`) to de-assert (go high) mid-burst even though your FPGA FIFO (`usbfifo_sync`) is also 1024 bytes. There are two real-world hardware scenarios where this happens:

1. USB Double-Buffering (512-Byte Sub-Buffers) & USB Host Delays

- In USB 2.0 Hi-Speed mode, the maximum Bulk packet size is **512 bytes**, and the FT233HP's 1024-byte RX Buffer (the buffer your FPGA writes into via `usbfifo_tx`) is managed by the **USB Protocol Engine and FIFO control** block to send data back to the Host PC when requested.
- If your FPGA previously sent a burst of data (for example, 600 bytes) into the FT233HP, and the PC operating system (Windows/Linux) is temporarily busy and hasn't issued a USB Bulk-IN request to read all of those bytes out of the FT233HP's 1 KB buffer yet, that 1 KB bufferinside the FT233HP is already partially full!
- Then, when the PC sends a new 600-byte packet to your FPGA (`usbfifo_rx`) and your `usbfifo_tx` immediately tries to write those 600 bytes back into the FT233HP, the FT233HP's 1024-byte RX buffer only has **424 bytes of free space left** (`1024 - 600 = 424`).
- After your FPGA writes the 424th byte, the FT233HP's 1024-byte buffer hits 100% capacity and **immediately drives `TXE#` (`i_txe_l`) high** right in the middle of your 600-byte burst (while your FPGA still has 176 bytes left to send and `i_empty == 0`)!

2. Concurrent USB Streaming (Continuous Transfers > 1024 Bytes)

- Even though the physical FIFO inside the FT233HP is 1024 bytes, if a user on the PC calls the FTDI D2XX driver function `FT_Write()` with a 4,096-byte array, the USB Protocol Engine streams 512-byte USB packets continuously into the 1024-byte FIFO as fast as your FPGA reads them out (up to 40 MBytes/sec).
- Similarly, when your FPGA writes data back into the FT233HP at **60 MBytes/sec** (1 byte every 16.67 ns on the 60 MHz `CLKOUT`), that burst rate (60 MB/s) is faster than the sustained USB 2.0 throughput (40 MB/s max). Whenever the USB host bus pauses between 512-byte USB frames, the FT233HP uses `TXE#` going high (`1`) to pause the FPGA for a few clock cycles until the USB engine frees up space in the 1 KB buffer.

Summary for Your Test Plan

- **In a single-shot loopback test ($\leq$ 512 or $\leq$ 1024 bytes):** If the FT233HP buffer starts completely empty and you send $\leq$ 512 bytes (1 USB Hi-Speed packet) from the PC to the FPGA and echo it back once, the FT233HP's 1024-byte buffer will **not** fill up, so `TXE#` (`i_txe_l`) will stay low (`0`) for the entire burst and your current `usbfifo_tx.v` will work cleanly.
- **In continuous streaming or back-to-back transfers (> 512 / > 1024 bytes):** Whenever the FT233HP's 1 KB buffer hasn't completely drained to the PC yet, `TXE#` (`i_txe_l`) will go high mid-burst to throttle the FPGA.

Will your FPGA verification test use single-packet bursts ($\leq$ 512 or 1024 bytes at a time), or continuous large data streams?

User prompt: I see your point. currently my testbench is single-shot loopback which means the size of fifo burst write is 1024 bytes. That's why my simulation looks clean I think. thanks for your recommendation. Just let me take note here so see if I need to revise RTL or not later. thanks.

Response: You're very welcome! That explains why your simulation waveforms look completely clean right now—in a single-shot 1024-byte loopback, `i_empty` will always be what ends the write burst, so your current `usbfifo_tx.v` handles the entire 1024-byte transfer without a hitch.

Keeping the current RTL for your initial hardware bring-up and only revisiting that `i_txe_l` mid-burst pause condition if you later move to continuous multi-kilobyte streaming is a very practical engineering approach.

Best of luck running synthesis and checking your timing report in Vivado—feel free to reach out anytime if you want to look over the timing slack results or anything else on the board bring-up!

User prompt: Now, would you please help me to make the summary for each modules? Need to cover each module's name, possible concern which I take notes while currently determine not to review. I would like to make sure we're on the same page before we go to timing constraints design and synthesis stage.

Response:

Module Name	Current Status / Action Taken	Noted Concerns (Deferred — Keep Original RTL for Now)	When to Revisit / How to Verify
`usbfifo_top`	**Keep or Optional Update:** Connects external FT233HP I/O pins and arbitrates the internal modules (`usbfifo_rx`, `usbfifo_tx`, `usbfifo_sync`).	1. **Idle Bus Contention Risk:** `assign io_data = (o_oe_l) ? o_tx_data : 8'hzz;` drives `io_data` whenever `o_oe_l == 1'b1` (including idle states), rather than only when `o_wr_l == 1'b0`. 2. **Critical Output Timing Path:** As noted in your header comment, the `o_rd_data` $\rightarrow$ `o_tx_data` $\rightarrow$ `io_data` path is a critical timing path to the external pins.	1. Revisit if bus contention or current spikes occur during hardware turnaround between read and write. 2. Verify slack during Vivado STA using `set_output_delay -max 8.000` (Table 17 `t12`).
`usbfifo_sync`	**Updated for BRAM Inference:** Separated the asynchronous reset (`negedge i_rst_n`) on `wr_ptr`/`rd_ptr` from the purely synchronous `mem` block (`posedge i_clk_60mhz`), and used an `initial` block so Vivado infers a dedicated Artix-7 **Block RAM (BRAM)** instead of thousands of flip-flops.	**Runtime Array Clearing:** Because hardware BRAM does not support clearing all 1,024 memory cells asynchronously via `i_rst_n`, resetting during runtime only resets `wr_ptr` and `rd_ptr` to `0` (`o_empty = 1`) rather than wiping the internal RAM contents.	No functional impact for FIFO operation, since `o_empty` blocks reads until new data overwrites old entries. Check the Vivado Synthesis Utilization Report to confirm `Block RAM Tile = 0.5` (one 18Kb BRAM).
`usbfifo_rx`	**Keep Original RTL (Option 1):** Verified that the state machine and signals (`o_oe_l`, `o_rd_l`, `valid_trigger`) functionally satisfy the FT233HP Figure 7 read waveform in single-shot loopback.	1. **Tight Setup Time on `valid_trigger`:** Uses raw input `!i_rxf_l` directly. Since FT233HP `CLKOUT` to `RXF#` (`t4`) is up to **9.0 ns** out of the **16.67 ns** period, only ~7.67 ns remains for routing and logic.	1 & 2. Check the Vivado Timing Summary Report after applying `set_input_delay -max 9.000` (`t4`) and `set_output_delay` (`t7 = 8.0 ns`, `t9 = 7.5 ns`). Only pipeline `i_rxf_l`/`i_rx_data` or register `o_oe_l`/`o_rd_l` if WNS < 0.

Module Name	Current Status / Action Taken	Noted Concerns (Deferred — Keep Original RTL for Now)	When to Revisit / How to Verify
		2. **Combinational Outputs (`o_oe_l`, `o_rd_l`):** Driven by an `always @(*)` block, which adds LUT delay before the pin and carries a slight risk of decoding glitches. 3. **Full FIFO Lock (`i_full` while `!i_rxf_l`):** If a burst ever exceeds 1,024 bytes (`i_full == 1`), `o_rx_active` stays high and prevents `usbfifo_tx` from draining the FIFO.	3. Only revisit (`if (i_full)` in `RX_DETE_RXF_N` + `!i_full` in `o_rx_active`) if testing continuous transfers $> 1,024$ bytes.
`usbfifo_tx`	**Keep Original RTL:** Verified that `o_wr_l` is cleanly registered and releases 1 clock cycle after `i_txe_l` or `i_empty` asserts, matching Figure 7 for single-shot $\leq 1,024$-byte loopback bursts.	1. **Mid-Burst `i_txe_l` De-assertion (1-Byte Pre-Fetch Over-Read):** Because `o_tx_rden` (`vaild_data`) pre-fetches 1 cycle ahead of `o_wr_l`, if the FT233HP fills up mid-burst and pulls `i_txe_l` high while `i_empty == 0`, 1 extra byte is popped from `usbfifo_sync` and ignored by the FT233HP. 2. **Input/Output Timing Budget:** `i_txe_l` (`t11` = 9.0 ns max) drives `vaild_data` combinationally, and `o_wr_l` requires an 8.0 ns setup time (`t14`) at the FT233HP.	1. Safe to ignore for single-shot $\leq 1,024$-byte loopback tests (where `i_empty` always ends the burst first). Revisit only if moving to continuous multi-kilobyte streaming. 2. Verify slack in Vivado STA using `set_input_delay -max 9.000` on `i_txe_l` and `set_output_delay -max 8.000` on `o_wr_l`.

We are aligned on all four modules. Whenever you are ready, we can move straight into designing your `.xdc` timing constraints and reviewing your Vivado synthesis and timing reports!

User prompt: Uploaded new RTL modules. please help to review them cycle-to-cycle again. Also, list a table above to me. Then, we should be able to go to design .xdc.

Response: Module-by-Module Review Summary Table

Module Name	Current Status / Action Taken (10112026 Version)	Cycle-to-Cycle Verification	Noted Concerns (Deferred — Keep Original RTL for Now)
`usbfifo_sync`	**Updated (`10012026 Modify`):** 1. Added `(* ram_style = "block" *)` attribute to `mem`. 2. Added `initial` block loop to initialize `mem` to `8'h00` at bitstream load. 3. Separated async-reset pointer blocks (`wr_ptr`,	**PASS:** • **Write:** On any rising clock edge where `wr_valid` (`i_wr_en && !o_full`) is `1`, `i_wr_data` is written into `mem[wr_addr]` and `wr_ptr` increments by 1 on that exact edge. • **Read:** On any rising edge where `rd_valid` (`i_rd_en && !o_empty`) is `1`, `mem[rd_addr]` is latched into `o_rd_data` (1-cycle synchronous BRAM read latency) and `rd_ptr` increments by 1.	1. **Cosmetic Note in `initial` Block:** You used non-blocking assignment (`mem[i] <= 8'h00;`) inside the `initial` loop. Vivado accepts this, though standard blocking assignment (`=`) is preferred in `initial` blocks. 2. **Runtime Reset:** As expected for hardware BRAM, asserting `i_rst_n` during runtime resets `wr_ptr` and `rd_ptr` to `0` (`o_empty = 1`) without wiping the physical BRAM array.

Module Name	Current Status / Action Taken (10112026 Version)	Cycle-to-Cycle Verification	Noted Concerns (Deferred — Keep Original RTL for Now)
	`rd_ptr`) from purely synchronous `posedge i_clk_60mhz` BRAM push/pop blocks.		
`usbfifo_rx`	**Updated (`Modify_10112026`):** 1. Added `if (i_full) nt_state = RX_DETE_RXF_N;` in `RX_DETE_RXF_N` so the FSM does not start a new read burst if the internal FIFO is full. 2. Added the commented-out option for `!i_full` in `o_rx_active` for future reference.	**PASS (for ≤ 1024-byte bursts):** • **Cycle 1 (`RX_DETE_RXF_N`):** Detects `!i_rxf_l && !i_tx_active` (when `!i_full`). • **Cycle 2 (`RX_DATA_PRE_OE_N`):** Drives `o_oe_l = 0` while `o_rd_l = 1` (satisfies the 1-cycle bus turnaround before `RD#` in Figure 7). • **Cycle 3..N (`RX_DATA_PRE_RD_N`):** Drives `o_oe_l = 0`, `o_rd_l = 0`, and `o_data_valid = 1`. Each clock edge writes 1 byte into `usbfifo_sync`. • **End of Burst:** The instant `i_rxf_l` goes high (`1`), `valid_trigger` drops to `0` combinationally so the trailing invalid byte is ignored. One cycle later (`r_rxf_n == 1`), `o_oe_l` and `o_rd_l` release to `1` and the FSM exits to `RX_RLS_DATA_RD_OE_N`.	1. **Setup Timing on `valid_trigger`:** Uses raw `!i_rxf_l` (`t4` = 9.0 ns max), leaving ~7.67 ns for FPGA routing/logic. Will verify via `.xdc` STA. 2. **Combinational Outputs (`o_oe_l`, `o_rd_l`):** Driven via `always @(*)`. Will check output delay slack in Vivado. 3. **Mid-Burst Full Condition (>1024 bytes):** In `RX_DATA_PRE_RD_N`, exit condition is `if (r_rxf_n)` and `!i_full` is commented out of `o_rx_active`. Safe for ≤ 1024 -byte tests.
`usbfifo_tx`	**Unchanged:** Retained registered `o_wr_l` output and 1-cycle pre-fetch logic (`en_vaild_data`).	**PASS (for single-shot loopback):** • **Cycle 1 (`TX_DETE_TX_L`):** When `!i_empty && !i_txe_l && !i_rx_active`, `vaild_data` (`o_tx_rden`) asserts combinationally. • **Edge 1 $\rightarrow$ Cycle 2 (`TX_DATA_WR_L`):** On the rising edge entering `TX_DATA_WR_L`, `usbfifo_sync` latches `Byte 0` onto `o_rd_data` (`o_tx_data`) at the exact same edge that `o_wr_l <= 1'b0` asserts. • **End of Burst (`i_empty == 1`):** When the last byte (e.g., `Byte 1023`) is popped onto `o_rd_data` at Edge K, `rd_ptr == wr_ptr` asserts `i_empty = 1`. During $K \rightarrow K+1$, `Byte 1023` and `o_wr_l = 0` remain valid on the pins while `nt_state` becomes `TX_RLS_WR_L`. At Edge $K+1$, FT233HP samples `Byte 1023` and `o_wr_l` releases to `1'b1`.	1. **Mid-Burst `i_txe_l` Pause Over-Read:** If `i_txe_l` ever goes high mid-burst while `i_empty == 0`, 1 pre-fetched byte is popped from `usbfifo_sync` and ignored by FT233HP. Safe to defer for single-shot ≤ 1024-byte loopback. 2. **Critical Output Path (`o_rd_data` $\rightarrow$ `io_data`):** Must meet 8.0 ns setup time (`t12`) at the FT233HP. Will verify via `.xdc` STA.
`usbfifo_top`	**Unchanged:**	**PASS:**	**Idle Bus Drive:** `assign io_data = (o_oe_l) ? o_tx_data : 8'hzz;` actively drives `o_tx_data` during idle

Module Name	Current Status / Action Taken (10112026 Version)	Cycle-to-Cycle Verification	Noted Concerns (Deferred — Keep Original RTL for Now)
	Connects `usbfifo_rx`, `usbfifo_tx`, and `usbfifo_sync`, and controls tri-state buffer `io_data`.	• When `o_oe_l == 0` (`usbfifo_rx` reading from FT233HP), `io_data` is tri-stated (`8'hzz`) so the FT233HP drives `i_rx_data`. • When `o_oe_l == 1` (`usbfifo_tx` writing to FT233HP), `io_data` drives `o_tx_data`.	states when `o_oe_l == 1`. Safe if FT233HP keeps `D[7:0]` tri-stated whenever `OE#` is high.

Cycle-by-Cycle Walkthrough of Your Updated RTL (End-to-End Loopback)

To be 100% certain of how your four uploaded files work together, here is the cycle-by-cycle trace across all four modules:

Phase 1: FT233HP $\rightarrow$ FPGA Read Burst (`usbfifo_rx` $\rightarrow$ `usbfifo_sync`)

1. **Clock Cycle C_0 (Detect Request):**
   - FT233HP asserts `i_rxf_l = 0` (`t4` after clock edge).
   - In `usbfifo_rx`, `st_state` is `RX_DETE_RXF_N` (`3'h1`). Because `i_full == 0` and `i_tx_active == 0`, `nt_state` evaluates to `RX_DATA_PRE_OE_N` (`3'h2`), and `o_rx_active` immediately asserts (`1'b1`) to lock out `usbfifo_tx`.
2. **Clock Cycle C_1 (Bus Turnaround — `OE#` Low):**
   - At Rising Edge C_1, `st_state <= RX_DATA_PRE_OE_N` and `r_rxf_n <= 1'b0`.
   - `o_oe_l` asserts (`1'b0`), which immediately switches `io_data` in `usbfifo_top` to `8'hzz`.
   - Seeing `OE#` low, the FT233HP turns on its output drivers and places **`Byte 0`** onto `io_data` (`t6` = 2.4 ns to 9.0 ns).
   - Note that `o_rd_l` is still `1'b1` and `valid_trigger` (`o_data_valid`) is `1'b0` during C_1, satisfying the mandatory 1-clock turnaround time before `RD#` goes low.
3. **Clock Cycle C_2 (First Data Sampled — `RD#` Low):**
   - At Rising Edge C_2, `st_state <= RX_DATA_PRE_RD_N` (`3'h3`).
   - Now `o_oe_l = 0`, `o_rd_l = 0`, and `valid_trigger` (`o_data_valid` / `i_wr_en`) becomes `1'b1`.
   - Throughout C_2, **`Byte 0`** sits stably on `i_rx_data` (`o_rx_data`).
4. **Rising Edge C_3 (`Byte 0` Written to BRAM, FT233HP Advances to `Byte 1`):**
   - **Inside `usbfifo_sync`:** Seeing `wr_valid == 1` at Rising Edge C_3, the new synchronous BRAM block executes `mem[0] <= Byte 0` and increments `wr_ptr` from `0` to `1` (de-asserting `o_empty` to `0`)!
   - **Inside FT233HP:** Seeing `o_rd_l == 0` at Rising Edge C_3, the FT233HP fetches **`Byte 1`** onto `io_data` (`t5` = 2.4 ns to 9.0 ns).
5. **End of Read Burst (After Last Byte is Latched at Edge C_{N+2}):**
   - Right after Rising Edge C_{N+2} (where the last valid byte was written into `mem[N-1]`), the FT233HP drives `i_rxf_l = 1` (`t4` = 1.4 ns to 9.0 ns) and outputs invalid data on `io_data`.
   - Because `valid_trigger = (st_state == RX_DATA_PRE_RD_N && !i_rxf_l && !i_full)`, `o_data_valid` drops to `0` **before** Rising Edge C_{N+3}, preventing the invalid data from being written to `mem[N]`.
   - At Rising Edge C_{N+3}, `r_rxf_n` latches `1'b1`, which releases `o_oe_l = 1` and `o_rd_l = 1` and transitions the FSM through `RX_RLS_DATA_RD_OE_N` $\rightarrow$ `RX_WAIT` $\rightarrow$ `RX_IDLE` $\rightarrow$ `RX_DETE_RXF_N`, dropping `o_rx_active` to `0`.

Phase 2: FPGA $\rightarrow$ FT233HP Write Burst (`usbfifo_sync` $\rightarrow$ `usbfifo_tx`)

1. **Clock Cycle W_0 (Pre-Fetch `Byte 0` in `TX_DETE_TX_L`):**

- As soon as `o_rx_active` drops to `0` (and `i_empty == 0`, `i_txe_l == 0`), `usbfifo_tx` (in `st_state == TX_DETE_TX_L`) sets `nt_state = TX_DATA_WR_L` and asserts `vaild_data` (`o_tx_rden = 1`) combinationally during W_0.

2. **Rising Edge W_1 (Launch `Byte 0` and `o_wr_l = 0` Simultaneously):**
   - **Inside `usbfifo_sync`:** Seeing `rd_valid == 1` at Rising Edge W_1, the synchronous BRAM read block executes `o_rd_data <= mem[0]` (`Byte 0`) and increments `rd_ptr` from `0` to `1`.
   - **Inside `usbfifo_tx`:** At that exact same Rising Edge W_1, `st_state <= TX_DATA_WR_L` and `o_wr_l <= 1'b0`.
   - **Result during $W_1 \rightarrow W_2$:** `Byte 0` (on `io_data`) and `o_wr_l = 0` are presented together on the external FT233HP pins for the full clock cycle, and at Rising Edge W_2 the FT233HP samples `Byte 0` while `usbfifo_sync` outputs `Byte 1` (`mem[1]`)!
3. **Final Byte of Write Burst (Rising Edge $W_N \rightarrow W_{N+1}$):**
   - At Rising Edge W_N, `usbfifo_sync` outputs the final byte (`mem[N-1]`) onto `o_rd_data` and increments `rd_ptr` to `N`.
   - Now `rd_ptr == wr_ptr`, so `o_empty` (`i_empty`) asserts (`1'b1`) right after Rising Edge W_N.
   - During $W_N \rightarrow W_{N+1}$:
     - `o_wr_l` is **still `1'b0`** and `mem[N-1]` is **still driven on `io_data`**.
     - Because `i_empty == 1`, `vaild_data` (`o_tx_rden`) drops to `0`, and `nt_state` becomes `TX_RLS_WR_L`.
   - At Rising Edge W_{N+1}:
     - The FT233HP samples the final byte `mem[N-1]` (since both `i_txe_l` and `o_wr_l` were `0` leading up to W_{N+1}).
     - Simultaneously at Edge W_{N+1}, `else if (nt_state == TX_RLS_WR_L) o_wr_l <= 1'b1;` releases `o_wr_l` to `1'b1`!

Ready for `.xdc` Timing Constraints Design

Your four RTL modules are clean, consistent, and ready for synthesis and timing analysis.

To design your complete `.xdc` file for the **Xilinx Artix-7 (`xc7a35tcpg`)**, what do you need included?

1. **Timing Constraints Only:** Clock definition (`i_clk_60mhz` at 16.67 ns) and Input/Output delays derived from **Table 17** (`t1`–`t15`).
2. **Physical Pin Assignments (`PACKAGE_PIN` & `IOSTANDARD LVCMOS33`):** Do you also have the specific FPGA pin numbers on your Artix-7 board for `i_clk_60mhz`, `i_rst_n`, `i_rxf_l`, `i_txe_l`, `o_oe_l`, `o_rd_l`, `o_wr_l`, and `io_data[7:0]`, or should we start with the timing constraints first?

User prompt: please list a table like what you did previously.

Response:

Module	Perspective	Identified Issue (in `1011_v01` RTL)	Recommended Solution / Action Plan
`usbfifo_top`	Function / Protocol	**Bus Contention Risk (Deferred):** The tri-state buffer `io_data` uses `o_oe_l` for direction control (`assign io_data = (o_oe_l) ? o_tx_data : 8'hzz;`), causing the FPGA to actively drive the bus during idle states.	**Keep for now.** If bus contention occurs during hardware testing, change the direction control to rely on the write strobe: `assign io_data = (!o_wr_l) ? o_tx_data : 8'hzz;`.
`usbfifo_top`	Timing	**Critical Output Path:** As noted in your code comment, the `o_rd_data` → `o_tx_data` → `io_data` path is critical and must meet the 8.0 ns setup time (`t12`) before the 60 MHz clock edge.	Control via `.xdc` timing constraints (`set_output_delay -max 8.000 [get_ports {io_data[*]}]`) and verify positive slack in the Vivado timing report.

Module	Perspective	Identified Issue (in `1011_v01` RTL)	Recommended Solution / Action Plan
`usbfifo_sync`	Syntax / Synthesis	**RESOLVED (BRAM Inference):** You separated the async-reset pointers from the synchronous `mem` read/write blocks and added `(* ram_style = "block" *)`. (Minor note: `mem[i] <= 8'h00;` inside the `initial` loop uses non-blocking `<=` instead of blocking `=`).	**Ready for synthesis.** Vivado will now infer a dedicated Artix-7 Block RAM (BRAM) instead of thousands of flip-flops. (Optionally change `<=` to `=` inside the `initial` block for strict Verilog style).
`usbfifo_rx`	Function / Protocol	**Mid-Burst Full Lock (Partially Resolved / Deferred):** You added `if(i_full) nt_state = RX_DETE_RXF_N;` to prevent starting a burst when full. However, if `i_full` asserts *mid-burst* (>1024 bytes), `RX_DATA_PRE_RD_N` waits for `r_rxf_n` and `o_rx_active` stays high (`&& !i_full` is commented out).	**Keep for now.** Safe for your single-shot ≤ 1024-byte loopback test. If you test >1024-byte bursts later, uncomment `&& !i_full` in `o_rx_active` and change line 91 to `if(r_rxf_n
`usbfifo_rx`	Syntax / Function	**Combinational Glitches (Deferred):** External control pins `o_oe_l` and `o_rd_l` are driven by a combinational `always @(*)` block, which adds LUT delay and risks minor decoding glitches.	**Keep for now.** Check output timing slack against `t7` (8.0 ns) and `t9` (7.5 ns) in Vivado. Convert to sequential `always @(posedge i_clk_60mhz)` only if timing fails or glitches appear on the scope.
`usbfifo_rx`	Timing	**Tight Setup Time (Deferred):** `valid_trigger` uses the raw `i_rxf_l` input directly. Since FT233HP `CLKOUT` to `RXF#` (`t4`) is up to 9.0 ns out of the 16.67 ns period, only ~7.67 ns remains for internal routing and logic.	Control via `.xdc` constraint (`set_input_delay -max 9.000 [get_ports i_rxf_l]`). If Vivado reports negative slack (WNS < 0), pipeline both `i_rxf_l` and `i_rx_data` with input registers.
`usbfifo_tx`	Function / Protocol	**Mid-Burst `i_txe_l` Pause Over-Read (Deferred):** Because `o_tx_rden` (`vaild_data`) pre-fetches 1 cycle ahead of `o_wr_l`, if the FT233HP fills up mid-burst and pulls `i_txe_l` high while `i_empty == 0`, 1 extra byte is popped from `usbfifo_sync` and ignored by the FT233HP.	**Keep for now.** Safe for your single-shot ≤ 1024-byte loopback test where `i_empty` always ends the burst first. Revisit only if moving to continuous multi-kilobyte streaming.
`usbfifo_tx`	Timing	**Strict Input / Output Delay:** `o_wr_l` requires an 8.0 ns setup time (`t14`), and `i_txe_l` has up to a 9.0 ns input delay (`t11`) driving `vaild_data` combinationally.	Control via `.xdc` constraints (`set_output_delay -max 8.000 [get_ports o_wr_l]` and `set_input_delay -max 9.000 [get_ports i_txe_l]`) and verify slack in Vivado.

User prompt: ok. thanks. Let's continue .xdc file. would you please provide you design on timing constraints based on datasheet and current all modules designs?

Response: To design the Xilinx Design Constraints (`.xdc`) file for your Artix-7 (`xc7a35tcpg`) FPGA, every timing parameter in **Table 17 (FT245 Synchronous FIFO Interface Signal Timings)** is mapped directly to the port names in your `usbfifo_top.v` module (`1011_v01`).

1. Mapping Table 17 Parameters to XDC Constraints

In a system-synchronous interface where the FT233HP supplies the 60 MHz `CLKOUT` (`i_clk_60mhz`) to the FPGA:

- **`set_input_delay` (FT233HP $\rightarrow$ FPGA):** Tells Vivado how much time has already been consumed by the FT233HP's internal Clock-to-Out delay (T_{co}) after the rising edge of `CLKOUT`.
    - `set_input_delay -max` = Maximum FT233HP T_{co} (+ PCB trace skew)
    - `set_input_delay -min` = Minimum FT233HP T_{co} (- PCB trace skew)

- **`set_output_delay` (FPGA → FT233HP):** Tells Vivado how much Setup Time (T_{su}) and Hold Time (T_h) the FT233HP requires around the rising edge of `CLKOUT`.
  - `set_output_delay -max` = FT233HP required Setup Time (T_{su})
  - `set_output_delay -min` = Negative of FT233HP required Hold Time ($-T_h$)

Top-Level Port (`usbfifo_top.v`)	Direction	Datasheet Parameter (Table 17)	Min	Nom	Max	XDC Constraint Value
`i_clk_60mhz`	Input Clock	**`t1`** (`CLKOUT` period) **`t2`/`t3`** (High/Low period)	—	16.67 ns 8.33 ns	—	`create_clock -period 16.670`
`i_rxf_l`	Input	**`t4`** (`CLKOUT` to `RXF#`)	1.4 ns	—	9.0 ns	`set_input_delay -max 9.000` `set_input_delay -min 1.400`
`i_txe_l`	Input	**`t11`** (`CLKOUT` to `TXE#`)	1.0 ns	—	9.0 ns	`set_input_delay -max 9.000` `set_input_delay -min 1.000`
`io_data[7:0]` (Read)	Inout (Input Mode)	**`t5`** (`CLKOUT` to read `DATA` valid)	2.4 ns	—	9.0 ns	`set_input_delay -max 9.000` `set_input_delay -min 2.400`
`o_oe_l`	Output	**`t7`** (`OE#` setup time) **`t8`** (`OE#` hold time)	8.0 ns 0.0 ns	—	16.67 ns —	`set_output_delay -max 8.000` `set_output_delay -min 0.000`
`o_rd_l`	Output	**`t9`** (`RD#` setup time to `CLKOUT`) **`t10`** (`RD#` hold time)	7.5 ns 0.0 ns	—	16.67 ns —	`set_output_delay -max 7.500` `set_output_delay -min 0.000`
`o_wr_l`	Output	**`t14`** (`WR#` setup time to `CLKOUT`) **`t15`** (`WR#` hold time)	8.0 ns 0.0 ns	—	16.67 ns —	`set_output_delay -max 8.000` `set_output_delay -min 0.000`

Top-Level Port (`usbfifo_top.v`)	Direction	Datasheet Parameter (Table 17)	Min	Nom	Max	XDC Constraint Value
`io_data[7:0]` (Write)	Inout (Output Mode)	**`t12`** (Write `DATA` setup time)	8.0 ns	—	16.67 ns	`set_output_delay -max 8.000`
		`t13` (Write `DATA` hold time)	0.0 ns		—	`set_output_delay -min 0.000`

2. Complete `.xdc` File (`usbfifo_timing.xdc`)

Copy this directly into your Vivado constraints file:

```
##############################################################################################
# FT233HP - FT245 Synchronous FIFO Interface Timing Constraints (DS_FT233HP Table 17)
# Target Device: AMD Xilinx Artix-7 (xc7a35tcpg)
# Top Module:    usbfifo_top
##############################################################################################

# ============================================================================
# 1. Primary System Clock Constraint (FT233HP 60 MHz CLKOUT)
#    Datasheet Table 17: t1 (Period) = 16.67 ns, t2 (High) = 8.33 ns, t3 (Low) = 8.33 ns
# ============================================================================
create_clock -period 16.670 -name clk_60mhz -waveform {0.000 8.335} [get_ports i_clk_60mhz]


# ============================================================================
# 2. Input Delay Constraints (FT233HP -> FPGA)
#    Referenced to rising edge of clk_60mhz
# ============================================================================

# RXF# Input Delay (Table 17 t4: Min = 1.4 ns, Max = 9.0 ns)
# Critical path in usbfifo_rx: i_rxf_l -> valid_trigger -> usbfifo_sync (wr_valid)
set_input_delay -clock [get_clocks clk_60mhz] -max 9.000 [get_ports i_rxf_l]
set_input_delay -clock [get_clocks clk_60mhz] -min 1.400 [get_ports i_rxf_l]

# TXE# Input Delay (Table 17 t11: Min = 1.0 ns, Max = 9.0 ns)
# Critical path in usbfifo_tx: i_txe_l -> vaild_data -> usbfifo_sync (rd_valid)
set_input_delay -clock [get_clocks clk_60mhz] -max 9.000 [get_ports i_txe_l]
set_input_delay -clock [get_clocks clk_60mhz] -min 1.000 [get_ports i_txe_l]

# Read DATA[7:0] Input Delay (Table 17 t5: Min = 2.4 ns, Max = 9.0 ns)
# Path: io_data[7:0] -> i_rx_data -> o_rx_data -> usbfifo_sync BRAM (i_wr_data)
set_input_delay -clock [get_clocks clk_60mhz] -max 9.000 [get_ports {io_data[*]}]
set_input_delay -clock [get_clocks clk_60mhz] -min 2.400 [get_ports {io_data[*]}]


# ============================================================================
# 3. Output Delay Constraints (FPGA -> FT233HP)
#    Referenced to rising edge of clk_60mhz
# ============================================================================

# OE# Output Delay (Table 17 t7 Setup = 8.0 ns, t8 Hold = 0.0 ns)
# Driven by combinational always@(*) in usbfifo_rx + controls io_data tri-state buffer in usbfifo_top
set_output_delay -clock [get_clocks clk_60mhz] -max 8.000 [get_ports o_oe_l]
set_output_delay -clock [get_clocks clk_60mhz] -min 0.000 [get_ports o_oe_l]

# RD# Output Delay (Table 17 t9 Setup = 7.5 ns, t10 Hold = 0.0 ns)
# Driven by combinational always@(*) in usbfifo_rx (depends on st_state, r_rxf_n, i_full)
set_output_delay -clock [get_clocks clk_60mhz] -max 7.500 [get_ports o_rd_l]
set_output_delay -clock [get_clocks clk_60mhz] -min 0.000 [get_ports o_rd_l]

# WR# Output Delay (Table 17 t14 Setup = 8.0 ns, t15 Hold = 0.0 ns)
# Driven by sequential flip-flop in usbfifo_tx
set_output_delay -clock [get_clocks clk_60mhz] -max 8.000 [get_ports o_wr_l]
set_output_delay -clock [get_clocks clk_60mhz] -min 0.000 [get_ports o_wr_l]
```

```
# Write DATA[7:0] Output Delay (Table 17 t12 Setup = 8.0 ns, t13 Hold = 0.0 ns)
# Critical path noted in usbfifo_top: usbfifo_sync BRAM (o_rd_data) -> o_tx_data -> io_data[7:0]
set_output_delay -clock [get_clocks clk_60mhz] -max 8.000 [get_ports {io_data[*]}]
set_output_delay -clock [get_clocks clk_60mhz] -min 0.000 [get_ports {io_data[*]}]


# ============================================================================
# 4. Asynchronous Reset False Path Constraint
#    Prevents Vivado from trying to time external button/reset pin i_rst_n to 60 MHz
# ============================================================================
set_false_path -from [get_ports i_rst_n]


# ============================================================================
# 5. Electrical I/O Standard (FT233HP VCCIO = +3.3V LVTTL / LVCMOS33)
#    Datasheet Section 5.2 Table 33 & 34: VCCIO = 3.3V, Drive Strength configurable
# ============================================================================
set_property IOSTANDARD LVCMOS33 [get_ports {i_clk_60mhz i_rst_n i_rxf_l i_txe_l o_oe_l o_rd_l o_wr_l}]
set_property IOSTANDARD LVCMOS33 [get_ports {io_data[*]}]
set_property SLEW FAST [get_ports {o_oe_l o_rd_l o_wr_l io_data[*]}]

# ============================================================================
# 6. Physical Pin Assignments (Uncomment and fill in your Artix-7 xc7a35tcpg pins)
#    NOTE: Ensure i_clk_60mhz is connected to a Clock-Capable pin (MRCC or SRCC)!
# ============================================================================
# set_property PACKAGE_PIN <PIN_NUM> [get_ports i_clk_60mhz]
# set_property PACKAGE_PIN <PIN_NUM> [get_ports i_rst_n]
# set_property PACKAGE_PIN <PIN_NUM> [get_ports i_rxf_l]
# set_property PACKAGE_PIN <PIN_NUM> [get_ports i_txe_l]
# set_property PACKAGE_PIN <PIN_NUM> [get_ports o_oe_l]
# set_property PACKAGE_PIN <PIN_NUM> [get_ports o_rd_l]
# set_property PACKAGE_PIN <PIN_NUM> [get_ports o_wr_l]
# set_property PACKAGE_PIN <PIN_NUM> [get_ports {io_data[0]}]
# set_property PACKAGE_PIN <PIN_NUM> [get_ports {io_data[1]}]
# set_property PACKAGE_PIN <PIN_NUM> [get_ports {io_data[2]}]
# set_property PACKAGE_PIN <PIN_NUM> [get_ports {io_data[3]}]
# set_property PACKAGE_PIN <PIN_NUM> [get_ports {io_data[4]}]
# set_property PACKAGE_PIN <PIN_NUM> [get_ports {io_data[5]}]
# set_property PACKAGE_PIN <PIN_NUM> [get_ports {io_data[6]}]
# set_property PACKAGE_PIN <PIN_NUM> [get_ports {io_data[7]}]
```

3. Important Technical Details on How These Constraints Interact with Your RTL

1. **Why `io_data[*]` Has Both `set_input_delay` AND `set_output_delay`:** Because `io_data[7:0]` is an `inout` bidirectional bus, Vivado automatically analyzes both directions independently:
   - It applies `set_input_delay` (`t5`) to the input path (`IBUF` → `i_rx_data` → `usbfifo_sync` BRAM write data input `i_wr_data`).
   - It applies `set_output_delay` (`t12`) to the output path (`usbfifo_sync` BRAM read output `o_rd_data` → `o_tx_data` → `OBUFT` data input) **and** to the tri-state enable path (`usbfifo_rx` `o_oe_l` → `OBUFT` `T` control pin)!
2. **Preventing the Combinational Loop / False Path on `io_data`:** In `usbfifo_top.v`, `io_data` feeds `i_rx_data` (which goes to the BRAM write port) and is driven by `o_tx_data` (which comes from the BRAM read port). Because the BRAM read and write ports are both synchronous (`posedge i_clk_60mhz`), there is no combinational feedthrough between `i_rx_data` and `o_tx_data`, so Vivado will cleanly separate the input timing check from the output timing check.
3. **Why `set_property SLEW FAST` Is Included:** Because the FT233HP requires an 8.0 ns setup time (`t7`, `t12`, `t14`) out of a 16.67 ns clock period, the FPGA only has **8.67 ns** to get signals from the internal flip-flops/BRAM out through the physical `OBUF`/`OBUFT` pad. Setting `SLEW FAST` on your `LVCMOS33` output buffers shaves ~1.0 to 1.5 ns off the pad transition time compared to the default `SLOW` slew rate.
4. **Watch the Clock Input Architecture (`BUFG` vs. `MMCM`):** When `i_clk_60mhz` enters the Artix-7 pin and routes through a standard `IBUF` + `BUFG` global clock buffer, the internal clock network delay inside the Artix-7 is roughly **~3.0 to 3.5 ns**.
   - **Effect on `set_input_delay` (Helps Setup!):** That ~3.2 ns internal clock insertion delay *delays* the clock arrival at your internal flip-flops, which actually **adds ~3.2 ns of setup margin** to `i_rxf_l`, `i_txe_l`, and `io_data` (helping your `valid_trigger` path pass setup!).
   - **Effect on `set_output_delay` (Hurts Setup!):** Conversely, because the internal clock arrives at the BRAM and `o_wr_l` flip-flop ~3.2 ns later than the external pin, that ~3.2 ns is **subtracted** from your 8.67 ns output budget

(`o_rd_data` → `io_data`, `o_wr_l`, `o_oe_l`, `o_rd_l`)!

- *Tip:* When you run Synthesis & Implementation with this `.xdc` file, if you see that **Input Paths pass easily** while **Output Paths (`o_rd_data` → `io_data` or `o_rd_l`) have negative setup slack** due to the `BUFG` clock insertion delay, you can instantiate a Xilinx **MMCM / PLL** (Clocking Wizard) in deskew mode (`SOURCE_SYNCHRONOUS` / phase-aligned feedback) to cancel out that ~3.2 ns `BUFG` insertion delay!

Go ahead and add this `.xdc` file to your Vivado project and run **Synthesis** (and **Implementation**, since true routing and I/O delays are only finalized after Place & Route). Share the **Utilization Report** and **Timing Summary Report** whenever it finishes!

User prompt: please generate a .xdc file to me and assume the pcb delay is 0.5ns max

Response: Including a **0.5 ns maximum PCB propagation delay** ($T_{\text{pcb_max}} = 0.5$ ns, $T_{\text{pcb_min}} = 0.0$ ns) affects the input and output delay equations differently because the 60 MHz `CLKOUT` (`i_clk_60mhz`) is sourced **from the FT233HP to the FPGA**:

1. **Input Delays (FT233HP → FPGA):** Because both the clock (`i_clk_60mhz`) and input signals (`i_rxf_l`, `i_txe_l`, `io_data` read) travel in the **same direction** (FT233HP → FPGA):
   - **Max Input Delay (Setup Check):** Worst case is max data trace delay (0.5 ns) minus min clock trace delay (0.0 ns):
     $$\text{Input Delay}_{\text{max}} = T_{\text{co_max}} + (T_{\text{pcb_data_max}} - T_{\text{pcb_clk_min}}) = T_{\text{co_max}} + 0.5 \text{ ns}$$
   - **Min Input Delay (Hold Check):** Worst case is min data trace delay (0.0 ns) minus max clock trace delay (0.5 ns):
     $$\text{Input Delay}_{\text{min}} = T_{\text{co_min}} + (T_{\text{pcb_data_min}} - T_{\text{pcb_clk_max}}) = T_{\text{co_min}} - 0.5 \text{ ns}$$
2. **Output Delays (FPGA → FT233HP):** Because the clock travels from FT233HP → FPGA ($T_{\text{pcb_clk}}$), and then the FPGA's output signals (`o_oe_l`, `o_rd_l`, `o_wr_l`, `io_data` write) travel **back** from FPGA → FT233HP ($T_{\text{pcb_data}}$), the signal experiences a **round-trip PCB delay** ($0.5 \text{ ns} + 0.5 \text{ ns} = 1.0$ ns max) relative to the clock at the FT233HP pin:
   - **Max Output Delay (Setup Check):**
     $$\text{Output Delay}_{\text{max}} = T_{\text{su}} + T_{\text{pcb_clk_max}} + T_{\text{pcb_data_max}} = T_{\text{su}} + 0.5 \text{ ns} + 0.5 \text{ ns} = T_{\text{su}} + 1.0 \text{ ns}$$
     (Note: If you meant 0.5 ns total round-trip skew rather than 0.5 ns per trace, use $+0.5$ ns instead; both values are documented in the comments below).
   - **Min Output Delay (Hold Check):**
     $$\text{Output Delay}_{\text{min}} = -T_{\text{h}} + (T_{\text{pcb_clk_min}} + T_{\text{pcb_data_min}}) = 0.0 \text{ ns}$$

Updated Timing Calculation Table (with 0.5 ns Max PCB Delay)

Top-Level Port (`usbfifo_top.v`)	Datasheet Spec (Table 17)	PCB Delay Adjustment ($T_{\text{pcb}} \leq 0.5$ ns)	Final `.xdc` Constraint Value
`i_clk_60mhz`	$t_1 = 16.67$ ns (60 MHz)	Reference clock at FPGA pin	`-period 16.670`
`i_rxf_l`	t_4: Min = 1.4 ns, Max = 9.0 ns	Max: $+0.5$ ns / Min: -0.5 ns	`-max 9.500` / `-min 0.900`
`i_txe_l`	t_{11}: Min = 1.0 ns, Max = 9.0 ns	Max: $+0.5$ ns / Min: -0.5 ns	`-max 9.500` / `-min 0.500`
`io_data[7:0]` (In)	t_5: Min = 2.4 ns, Max = 9.0 ns	Max: $+0.5$ ns / Min: -0.5 ns	`-max 9.500` / `-min 1.900`
`o_oe_l`	t_7 (T_{su}) = 8.0 ns, t_8 (T_{h}) = 0.0 ns	Max: $+0.5$ ns trace ($+1.0$ ns round-trip)	`-max 8.500` *(or `9.000` round-trip)* / `-min 0.000`
`o_rd_l`	t_9 (T_{su}) = 7.5 ns, t_{10} (T_{h}) = 0.0 ns	Max: $+0.5$ ns trace ($+1.0$ ns round-trip)	`-max 8.000` *(or `8.500` round-trip)* / `-min 0.000`

Top-Level Port (`usbfifo_top.v`)	Datasheet Spec (Table 17)	PCB Delay Adjustment ($T_{\text{pcb}} \leq 0.5$ ns)	Final `.xdc` Constraint Value
`o_wr_l`	t_{14} (T_{su}) = 8.0 ns, t_{15} (T_{h}) = 0.0 ns	Max: $+0.5$ ns trace ($+1.0$ ns round-trip)	`-max 8.500` *(or `9.000` round-trip)* / `-min 0.000`
`io_data[7:0]` **(Out)**	t_{12} (T_{su}) = 8.0 ns, t_{13} (T_{h}) = 0.0 ns	Max: $+0.5$ ns trace ($+1.0$ ns round-trip)	`-max 8.500` *(or `9.000` round-trip)* / `-min 0.000`

Complete `.xdc` File (`usbfifo_top.xdc`)

Save the block below as `usbfifo_top.xdc` and add it to your Vivado project's **Constraints** set:

```
##############################################################################################
# File Name:     usbfifo_top.xdc
# Target Device: AMD Xilinx Artix-7 (xc7a35t)
# Top Module:    usbfifo_top
# Description:   FT233HP Synchronous 245 FIFO Timing Constraints (Table 17)
#                Includes 0.5 ns max PCB trace delay budget
##############################################################################################

# ============================================================================
# 1. System Clock Definition (FT233HP 60 MHz CLKOUT)
#    Datasheet Table 17: t1 = 16.67 ns (60 MHz), t2/t3 = 8.33 ns
# ============================================================================
create_clock -period 16.670 -name clk_60mhz -waveform {0.000 8.335} [get_ports i_clk_60mhz]


# ============================================================================
# 2. Input Delay Constraints (FT233HP -> FPGA)
#    Assumes T_pcb_max = 0.5 ns, T_pcb_min = 0.0 ns
#    Max Input Delay = T_co_max + 0.5 ns
#    Min Input Delay = T_co_min - 0.5 ns
# ============================================================================

# RXF# Input Delay (Table 17 t4: Min = 1.4 ns, Max = 9.0 ns)
# Max = 9.0 + 0.5 = 9.500 ns | Min = 1.4 - 0.5 = 0.900 ns
set_input_delay -clock [get_clocks clk_60mhz] -max 9.500 [get_ports i_rxf_l]
set_input_delay -clock [get_clocks clk_60mhz] -min 0.900 [get_ports i_rxf_l]

# TXE# Input Delay (Table 17 t11: Min = 1.0 ns, Max = 9.0 ns)
# Max = 9.0 + 0.5 = 9.500 ns | Min = 1.0 - 0.5 = 0.500 ns
set_input_delay -clock [get_clocks clk_60mhz] -max 9.500 [get_ports i_txe_l]
set_input_delay -clock [get_clocks clk_60mhz] -min 0.500 [get_ports i_txe_l]

# Read DATA[7:0] Input Delay (Table 17 t5: Min = 2.4 ns, Max = 9.0 ns)
# Max = 9.0 + 0.5 = 9.500 ns | Min = 2.4 - 0.5 = 1.900 ns
set_input_delay -clock [get_clocks clk_60mhz] -max 9.500 [get_ports {io_data[*]}]
set_input_delay -clock [get_clocks clk_60mhz] -min 1.900 [get_ports {io_data[*]}]


# ============================================================================
# 3. Output Delay Constraints (FPGA -> FT233HP)
#    Assumes 0.5 ns PCB delay budget added to FT233HP Setup requirement:
#    Max Output Delay = T_su + 0.5 ns (Note: use +1.0 ns if accounting for 0.5 ns
#                       CLKOUT trace + 0.5 ns DATA/CTRL return trace round-trip)
#    Min Output Delay = -T_h = 0.000 ns
# ============================================================================

# OE# Output Delay (Table 17 t7 Setup = 8.0 ns, t8 Hold = 0.0 ns)
# Max = 8.0 + 0.5 = 8.500 ns | Min = 0.000 ns
set_output_delay -clock [get_clocks clk_60mhz] -max 8.500 [get_ports o_oe_l]
set_output_delay -clock [get_clocks clk_60mhz] -min 0.000 [get_ports o_oe_l]

# RD# Output Delay (Table 17 t9 Setup = 7.5 ns, t10 Hold = 0.0 ns)
# Max = 7.5 + 0.5 = 8.000 ns | Min = 0.000 ns
set_output_delay -clock [get_clocks clk_60mhz] -max 8.000 [get_ports o_rd_l]
set_output_delay -clock [get_clocks clk_60mhz] -min 0.000 [get_ports o_rd_l]
```

```
# WR# Output Delay (Table 17 t14 Setup = 8.0 ns, t15 Hold = 0.0 ns)
# Max = 8.0 + 0.5 = 8.500 ns | Min = 0.000 ns
set_output_delay -clock [get_clocks clk_60mhz] -max 8.500 [get_ports o_wr_l]
set_output_delay -clock [get_clocks clk_60mhz] -min 0.000 [get_ports o_wr_l]

# Write DATA[7:0] Output Delay (Table 17 t12 Setup = 8.0 ns, t13 Hold = 0.0 ns)
# Max = 8.0 + 0.5 = 8.500 ns | Min = 0.000 ns
set_output_delay -clock [get_clocks clk_60mhz] -max 8.500 [get_ports {io_data[*]}]
set_output_delay -clock [get_clocks clk_60mhz] -min 0.000 [get_ports {io_data[*]}]


# ============================================================================
# 4. Timing Exceptions
#    Asynchronous active-low system reset (i_rst_n)
# ============================================================================
set_false_path -from [get_ports i_rst_n]


# ============================================================================
# 5. I/O Electrical Standards & Slew Rate Optimization
#    3.3V CMOS I/O Bank with FAST slew rate to minimize output buffer delay (T_cko)
# ============================================================================
set_property IOSTANDARD LVCMOS33 [get_ports {i_clk_60mhz i_rst_n i_rxf_l i_txe_l o_oe_l o_rd_l o_wr_l}]
set_property IOSTANDARD LVCMOS33 [get_ports {io_data[*]}]
set_property SLEW FAST [get_ports {o_oe_l o_rd_l o_wr_l io_data[*]}]


# ============================================================================
# 6. Physical Pin Assignments (Uncomment and replace with your board schematic pins)
# ============================================================================
# set_property PACKAGE_PIN <PIN_CLK>  [get_ports i_clk_60mhz]
# set_property PACKAGE_PIN <PIN_RST>  [get_ports i_rst_n]
# set_property PACKAGE_PIN <PIN_RXF>  [get_ports i_rxf_l]
# set_property PACKAGE_PIN <PIN_TXE>  [get_ports i_txe_l]
# set_property PACKAGE_PIN <PIN_OE>   [get_ports o_oe_l]
# set_property PACKAGE_PIN <PIN_RD>   [get_ports o_rd_l]
# set_property PACKAGE_PIN <PIN_WR>   [get_ports o_wr_l]
# set_property PACKAGE_PIN <PIN_D0>   [get_ports {io_data[0]}]
# set_property PACKAGE_PIN <PIN_D1>   [get_ports {io_data[1]}]
# set_property PACKAGE_PIN <PIN_D2>   [get_ports {io_data[2]}]
# set_property PACKAGE_PIN <PIN_D3>   [get_ports {io_data[3]}]
# set_property PACKAGE_PIN <PIN_D4>   [get_ports {io_data[4]}]
# set_property PACKAGE_PIN <PIN_D5>   [get_ports {io_data[5]}]
# set_property PACKAGE_PIN <PIN_D6>   [get_ports {io_data[6]}]
# set_property PACKAGE_PIN <PIN_D7>   [get_ports {io_data[7]}]
```

Checklist When You Run Synthesis & Implementation in Vivado

1. **Check the Utilization Report (`Report Utilization`):** Verify that `usbfifo_sync` maps to **0.5 Block RAM Tile** (`RAMB18E1`) rather than hundreds of Slice LUTs/Flip-Flops.
2. **Check the Timing Summary (`Report Timing Summary`):**
   - Look at **Intra-Clock Paths (`clk_60mhz`)** for internal register-to-register and register-to-BRAM timing.
   - Look at **Input-to-Register** paths (specifically `i_rxf_l` → `usbfifo_sync` write enable/pointers and `i_txe_l` → `usbfifo_sync` read enable/pointers) under the `9.500 ns` input delay.
   - Look at **Register-to-Output** paths (specifically `usbfifo_sync/o_rd_data_reg` → `io_data[*]` and `usbfifo_rx` combinational `o_oe_l` / `o_rd_l`) under the `8.500 ns` / `8.000 ns` output delays.
